

Resumen del taller 1 de Algoritmo y Lógica Computacional

Taller: Algoritmos y Lógica Computacional

Desarrollo de Lógica de Programación con JavaScript

SENA · Ingeniería de Software

Temas: Variables · Operadores · Estructuras de Control · Decisiones · Ciclos

Adaptado para Fedora Linux + Node.js

"La lógica te llevará de A a B. La imaginación te llevará a todas partes." — Albert Einstein

Tabla de Contenidos

-  [Objetivo y Criterios de Evaluación](#)
-  [Setup en Fedora Linux](#)
-  [Ejercicio 1 — Lavandería Express](#)
-  [Ejercicio 2 — Burger Palace](#)
-  [Ejercicio 3 — Alcaldía de Armenia](#)
-  [Ejercicio 4 — ParquiFácil](#)
-  [Ejercicio 5 — BiblioTech](#)
-  [Recomendaciones Finales](#)

Objetivo y Criterios de Evaluación

Este taller tiene como propósito fortalecer tu capacidad de análisis y resolución de problemas mediante la programación en JavaScript. A través de **5 ejercicios prácticos basados en situaciones reales**, aplicarás los conceptos fundamentales de: declaración de variables, operadores, estructuras de decisión y estructuras de repetición.

 **Importante:** No se proporciona código. Tú eres el arquitecto de la solución. Lee con atención, planifica tu lógica y luego codifica.

Requisitos previos

Debes manejar: `console.log()`, `prompt()` / `readline` en Node.js, `let` / `const`, operadores (`+`, `-`, `*`, `/`, `%`), `if` / `else`, operador ternario (`?:`), `while`, `do...while`, `for`, `parseInt()` / `parseFloat()`.

Criterios de evaluación

Criterio	Descripción
Variables	Correcta declaración y uso
Estructuras de control	Uso adecuado según el problema
Lógica	Clara, funcional y sin errores
Buenas prácticas	Nombres descriptivos, indentación correcta
Ejecución	El programa compila y muestra los resultados esperados

Setup en Fedora Linux

Estos ejercicios se ejecutan con **Node.js**. No necesitas el navegador.

```
# Instalar Node.js en Fedora
sudo dnf install nodejs -y

# Verificar versión
node --version    # v20.x.x o superior
npm --version

# Ejecutar un ejercicio
node ejercicio1_lavanderia.js
```

Manejo de entrada en Node.js

En el navegador se usa `prompt()`, pero en Node.js necesitas `readline`:

```
// Plantilla base para TODOS los ejercicios
const readline = require("readline");

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

// Función helper para pedir datos (evita el callback hell)
function preguntar(texto) {
  return new Promise((resolve) => {
    rl.question(texto, (respuesta) => resolve(respuesta));
  });
}
```

```
// Ejemplo de uso con async/await
async function main() {
  const nombre = await preguntar("¿Cómo te llamas? ");
  const edadStr = await preguntar("¿Cuántos años tienes? ");
  const edad = parseInt(edadStr);

  console.log(`Hola ${nombre}, tienes ${edad} años.`);
  rl.close();
}

main();
```

💡 Copia esta plantilla base al inicio de cada ejercicio y construye tu `main()` adentro. Siempre llama `rl.close()` al final para que el programa termine correctamente.

🧺 Ejercicio 1 — Lavandería Express

🕒 **Ciclo principal:** `for` | **Decisión:** `if/else` + operador ternario

Contexto del problema

La empresa **Lavandería Express** alquila lavadoras industriales por horas. El sistema debe registrar varios clientes y generar un resumen del día.

Regla	Detalle
Costo base	\$5.000 COP por hora
Descuento	30% sobre el total si el cliente alquila más de 12 horas

Instrucciones paso a paso

- Solicita al usuario cuántos clientes va a registrar en el día.
- Usa un ciclo `for` para iterar por cada cliente.
- Para cada cliente, solicita su **nombre** y la **cantidad de horas** que desea alquilar.
- Calcula el costo total: `horas × $5.000`.
- Usa `if/else` para verificar si las horas son mayores a 12. Si es así, aplica el descuento del 30%.
- Usa el **operador ternario** para mostrar `"CON DESCUENTO"` o `"SIN DESCUENTO"`.
- Muestra en consola: nombre, horas, costo sin descuento, valor del descuento y total a pagar.
- Lleva un **acumulador** que sume todos los totales a pagar.
- Al finalizar el ciclo, muestra el resumen del día.

Variables sugeridas

```
const costoPorHora = 5000;
let cantidadClientes, nombreCliente, horasAlquiler;
let costoTotal, descuento, totalPagar;
let acumuladorIngresos = 0;
let contadorDescuentos = 0;
```

Fórmulas clave

```
costoTotal      = horas × 5000
descuento       = costoTotal × 0.30    (solo si horas > 12)
totalPagar      = costoTotal - descuento
acumuladorIngresos += totalPagar
```

Salida esperada en consola

```
--- CLIENTE 1: María López ---
Horas alquiladas: 15
Subtotal:          $75.000
Descuento (30%):   $22.500  -  CON DESCUENTO
Total a pagar:     $52.500

=== RESUMEN DEL DÍA ===
Clientes atendidos:    5
Ingreso total:         $310.000
Clientes con descuento: 2
```

Preguntas guía para pensar antes de programar

- ¿En qué momento del ciclo verificas si hay descuento?
- ¿El acumulador debe estar **dentro** o **fuera** del `for`? ¿Por qué?
- ¿Qué pasa si el usuario ingresa 12 horas exactas? ¿Aplica descuento?



Ejercicio 2 — Burger Palace



Ciclo principal: `do...while` | Decisión: `if/else if/else`

Contexto del problema

Burger Palace ofrece 3 combos. El sistema toma pedidos de forma continua hasta que el usuario decide finalizar.

Opción	Combo	Precio
1	Clásica (hamburguesa + papas + gaseosa)	\$15.000
2	Doble Poder (doble + papas grandes + gaseosa)	\$22.000
3	Mega Fest (triple + papas + malteada + postre)	\$35.000
4	Finalizar pedido	—

Instrucciones paso a paso

1. Muestra el menú en consola con los 3 combos y la opción 4 para finalizar.
2. Usa un ciclo `do...while` — el menú se muestra **al menos una vez** y se repite mientras el usuario no seleccione la opción 4.
3. Dentro del ciclo, solicita al usuario que seleccione una opción (1, 2, 3 o 4).
4. Usa `if / else if / else` para determinar qué combo fue seleccionado y asignar su precio.
5. Si el usuario ingresa un número fuera de rango, muestra "Opción no válida" y vuelve a pedir.
6. Si la selección es válida (1, 2 o 3), solicita la **cantidad** de ese combo.
7. Calcula el subtotal: `precio × cantidad` y acumúlalo al total general.
8. Muestra: combo seleccionado, cantidad, subtotal y total acumulado hasta el momento.
9. Lleva un contador del total de combos pedidos y contadores individuales por tipo.
10. Al seleccionar opción 4, muestra el resumen final.

Variables sugeridas

```
let opcion, cantidadCombo, precioCombo, subtotal;
let totalCuenta = 0;
let totalCombos = 0;
let contadorCombo1 = 0;
let contadorCombo2 = 0;
let contadorCombo3 = 0;
```

Fórmulas clave

```
subtotal      = precioCombo × cantidadCombo
totalCuenta += subtotal
totalCombos += cantidadCombo
```

Salida esperada en consola

```
===== BURGER PALACE =====
1. Clásica      — $15.000
```

- 2. Doble Poder – \$22.000
- 3. Mega Fest – \$35.000
- 4. Finalizar pedido

Seleccione combo: 2
Cantidad: 3
Subtotal: \$66.000
Total acumulado: \$66.000

=== CUENTA FINAL ===
Combos Clásica: 2
Combos Doble Poder: 3
Combos Mega Fest: 1
Total combos: 6
TOTAL A PAGAR: \$161.000

Preguntas guía

- ¿Por qué `do...while` y no simplemente `while` ? ¿Qué diferencia hace en este caso?
- ¿Dónde exactamente sales del ciclo cuando el usuario elige la opción 4?
- ¿Cómo manejas el caso donde el usuario ingresa letras en vez de números?



Ejercicio 3 — Alcaldía de Armenia



Ciclo principal: `for` | Decisión: `if/else if/else` + operador ternario

Contexto del problema

La **Alcaldía de Armenia** implementa un programa de bienestar para el adulto mayor. El beneficio se calcula sobre el salario mínimo vigente de **\$1.300.000 COP**.

Rango de edad	Beneficio	Categoría
60 – 80 años (inclusive)	12% del salario mínimo	Adulto Mayor
Mayor de 80 años	15% del salario mínimo	Adulto Mayor Senior
Menor de 60 años	No aplica	—

Instrucciones paso a paso

1. Solicita al usuario cuántas personas va a registrar.
2. Usa un ciclo `for` para iterar por cada persona.
3. Para cada persona, solicita: **nombre completo** y **edad**.

4. Usa `if / else if / else` para clasificar: 60–80 años → 12%, mayor de 80 → 15%, menor de 60 → no aplica.
5. Usa el **operador ternario** para asignar la categoría: "Adulto Mayor" o "Adulto Mayor Senior".
6. Muestra: nombre, edad, categoría, porcentaje aplicado y valor del subsidio.
7. Lleva contadores: beneficiarios de 60–80, beneficiarios mayores de 80, personas que no aplican.
8. Lleva un acumulador del presupuesto total.
9. Al finalizar, muestra el informe completo de la alcaldía.

Variables sugeridas

```
const salarioMinimo = 1300000;  
let cantidadPersonas, nombre, edad, porcentaje, subsidio, categoria;  
let contBeneficiarios60_80 = 0;  
let contBeneficiariosMayor80 = 0;  
let contNoAplica = 0;  
let presupuestoTotal = 0;
```

Fórmulas clave

```
subsidio = salarioMinimo × 0.12    (si 60 <= edad <= 80)  
subsidio = salarioMinimo × 0.15    (si edad > 80)  
presupuestoTotal += subsidio
```

Salida esperada en consola

```
--- PERSONA 1: Carmen Rodríguez ---  
Edad:      72 años  
Categoría: Adulto Mayor  
Subsidio (12%): $156.000  
  
=== INFORME ALCALDÍA DE ARMENIA ===  
Total registrados:      10  
Beneficiarios (60-80 años): 6  – Subsidio: $156.000 c/u  
Beneficiarios (>80 años):  2  – Subsidio: $195.000 c/u  
No aplican:             2  
PRESUPUESTO TOTAL: $1.326.000
```

Preguntas guía

- ¿Cómo escribes la condición `60 <= edad <= 80` correctamente en JavaScript? (Pista: no como en matemáticas)
- ¿Qué pasa si alguien tiene exactamente 60 años? ¿Y exactamente 80?

- El operador ternario puede asignar solo dos valores. ¿Qué hacés con los tres casos posibles?

Ejercicio 4 — ParquiFácil

 **Ciclo principal:** `while` | **Decisión:** `if/else if/else` + operador ternario

Contexto del problema

ParquiFácil es un parqueadero en el centro de Armenia con tarifas diferenciadas y descuento por jornada completa.

Tipo de vehículo	Tarifa por hora
1 — Moto	\$2.000
2 — Carro	\$4.000
3 — Camioneta / SUV	\$6.000

Regla adicional	Detalle
Permanencia > 8 horas	20% de descuento sobre el total calculado

Instrucciones paso a paso

1. Muestra un menú: 1. Registrar vehículo / 2. Cerrar jornada .
2. Usa un ciclo `while` que se repita mientras el usuario no seleccione "Cerrar jornada".
3. Al registrar, solicita: **tipo de vehículo** (1, 2 o 3) y **horas de permanencia**.
4. Usa `if / else if / else` para asignar la tarifa según el tipo. Si el tipo es inválido, muestra error y vuelve al menú.
5. Calcula el costo total: `tarifa × horas` .
6. Usa `if` para verificar si las horas superan 8. Si es así, aplica 20% de descuento.
7. Usa el **operador ternario** para mostrar: `"TARIFA POR HORAS"` o `"TARIFA DÍA COMPLETO (20% desc.)"` .
8. Muestra el detalle del vehículo: tipo, horas, subtotal, descuento y total a pagar.
9. Lleva contadores individuales por tipo de vehículo.
10. Al cerrar jornada, muestra el reporte completo con el promedio de horas.

Variables sugeridas

```
let opcionMenu, tipoVehiculo, horasPermanencia;
let tarifaHora, costoTotal, descuento, totalPagar;
let contMotos = 0, contCarros = 0, contCamionetas = 0;
```

```
let ingresoTotal = 0;
let sumaHoras = 0;
let totalVehiculos = 0;
```

Fórmulas clave

```
costoTotal      = tarifaHora × horas
descuento       = costoTotal × 0.20      (solo si horas > 8)
totalPagar      = costoTotal - descuento
promedioHoras   = sumaHoras / totalVehiculos
```

Salida esperada en consola

```
--- VEHÍCULO REGISTRADO ---
Tipo:           Carro
Horas:          10
Subtotal:       $40.000
Descuento (20%): $8.000  -  TARIFA DÍA COMPLETO
Total:          $32.000

=== CIERRE DE JORNADA ===
Motos:          8
Carros:         15
Camionetas:     5
Total vehículos: 28
Ingreso total:  $624.000
Promedio permanencia: 5.2 horas
```

Preguntas guía

- ¿Por qué `while` es más apropiado que `for` aquí? ¿Cuál es la diferencia conceptual?
- ¿El contador `totalVehiculos` debe incrementarse antes o después de validar el tipo de vehículo?
- ¿Qué pasa si el parqueadero no registró ningún vehículo al cierre? ¿Cómo evitas dividir por cero en el promedio?

Ejercicio 5 — BiblioTech

 **Ciclos:** `for` externo + `for` anidado + `while` de validación | **Decisión:** `if/else` + `if/else` + ternario

Contexto del problema

La biblioteca pública **BiblioTech** de Armenia controla devoluciones de libros y calcula multas por retraso.

Regla	Detalle
Máximo de libros por usuario	3 libros
Préstamo estándar sin costo	7 días
Multa por retraso	\$1.500 por día por libro
Multa adicional si retraso > 15 días	\$10.000 adicionales por libro

Instrucciones paso a paso

1. Solicita cuántos usuarios van a realizar devoluciones hoy.
2. Usa un ciclo `for` para iterar por cada usuario.
3. Para cada usuario, solicita: **nombre y cantidad de libros** que devuelve. Valida con un ciclo `while` que la cantidad no exceda 3.
4. Para cada libro del usuario, usa un **ciclo `for` anidado**: solicita los días que tuvo el libro. Calcula los días de retraso: `días - 7` (solo si es mayor a 7).
5. Usa `if / else if / else`:
 - Sin retraso → multa: `$0`
 - Retraso de 1 a 15 días → multa: `diasRetraso × $1.500`
 - Retraso mayor a 15 días → multa: `(diasRetraso × $1.500) + $10.000`
6. Usa el **operador ternario** para clasificar al usuario: `"PUNTUAL"` o `"CON RETRASO"`.
7. Muestra el detalle por libro: días de préstamo, días de retraso, multa.
8. Suma la multa total del usuario (todos sus libros) y muéstrala.
9. Lleva contadores: libros devueltos en total, libros con retraso, libros puntuales.
10. Al finalizar, muestra el resumen del día.

Variables sugeridas

```
const multaDiaria    = 1500;
const multaAdicional = 10000;
let cantidadUsuarios, nombreUsuario, cantidadLibros;
let diasPrestamo, diasRetraso, multaLibro, multaUsuario;
let totalMultas      = 0;
let totalLibros      = 0;
let librosConRetraso = 0;
let librosPuntuales  = 0;
```

Fórmulas clave

```
diasRetraso = diasPrestamo - 7           (solo si diasPrestamo > 7)
multaLibro  = diasRetraso × 1500         (si retraso entre 1 y 15 días)
multaLibro  = (diasRetraso × 1500) + 10000 (si retraso > 15 días)
multaUsuario = suma de multas de todos sus libros
```

Salida esperada en consola

```
--- USUARIO 1: Andrés Gómez ---
Libros devueltos: 2
  Libro 1: 7 días – Sin retraso – Multa: $0
  Libro 2: 12 días – 5 días retraso – Multa: $7.500
Multa total usuario: $7.500 – CON RETRASO

=== RESUMEN BIBLIOTECHECH ===
Usuarios atendidos: 8
Total libros:      19
Libros puntuales:  11
Libros con retraso: 8
MULTAS RECAUDADAS: $127.500
```

Preguntas guía

- ¿Cómo funciona el **ciclo anidado**? ¿El ciclo interno se reinicia para cada usuario?
- ¿Dónde reinicias `multaUsuario` a 0? ¿Qué pasa si no lo hacés?
- ¿Cómo validas que la cantidad de libros no exceda 3? ¿Qué tipo de ciclo usarías para pedir el dato de nuevo si es inválido?

✓ Recomendaciones Finales

Antes de programar

1. Lee el ejercicio completo **al menos dos veces**.
2. Identifica las tres partes de todo programa:
 - **Entradas:** datos que pedís al usuario
 - **Proceso:** cálculos y decisiones
 - **Salidas:** lo que mostrás en consola
3. Dibuja un diagrama de flujo o escribe pseudocódigo antes de tocar el teclado.

💬 "Pensar primero, codificar después."

Al programar

- Usa nombres de variables descriptivos: `totalPagar`, no `tp`; `horasAlquiler`, no `h`.

- Indenta tu código correctamente (usa 2 o 4 espacios, elige uno y sé consistente).
- Prueba con **tres tipos de valores**:
 - Valores normales (caso típico)
 - Valores límite (exactamente 12 horas, exactamente 60 años, exactamente 7 días)
 - Valores extremos (0 horas, 100 años, 50 días de retraso)
- Comenta las secciones importantes de tu código.

Resumen de qué ciclo usar en cada ejercicio

Ejercicio	Ciclo principal	¿Por qué ese ciclo?
1 — Lavandería	<code>for</code>	Sabes de antemano cuántos clientes hay
2 — Burger Palace	<code>do...while</code>	El menú debe mostrarse al menos una vez
3 — Alcaldía	<code>for</code>	Sabes cuántas personas vas a registrar
4 — ParquiFácil	<code>while</code>	No sabes cuántos vehículos llegarán
5 — BiblioTech	<code>for + for anidado</code>	Iterás usuarios y dentro iterás libros

Entrega

Sube 5 archivos `.js` a tu repositorio de GitHub con estos nombres exactos:

```
ejercicio1_lavanderia.js
ejercicio2_hamburguesas.js
ejercicio3_alcaldia.js
ejercicio4_parqueadero.js
ejercicio5_biblioteca.js
```

```
# Comandos de Git para subir tu taller
git add .
git commit -m "feat: taller algoritmos y logica computacional JS"
git push origin main
```

Cada archivo debe ejecutarse sin errores con `node nombreArchivo.js`.

"El código es poesía escrita en lógica."

¡Éxitos en tu taller, futuro ingeniero de software! 🎓

```
Dataview (inline field '===== BURGER PALACE =====
```

```
1. Clásica          - $15.000
2. Doble Poder      - $22.000
3. Mega Fest        - $35.000
4. Finalizar pedido
```

```
Seleccione combo: 2
```

```
Cantidad: 3
```

```
Subtotal:          $66.000
```

```
Total acumulado:  $66.000
```

```
=== CUENTA FINAL ===
```

```
Combos Clásica:      2
```

```
Combos Doble Poder:  3
```

```
Combos Mega Fest:    1
```

```
Total combos:        6
```

```
TOTAL A PAGAR:        $161.000'): Error:
```

```
-- PARSING FAILED -----
----
```

```
> 1 | ===== BURGER PALACE =====
    | ^
    2 | 1. Clásica          - $15.000
    3 | 2. Doble Poder      - $22.000
```

Expected one of the following:

```
('(', 'null', boolean, date, duration, file link, list ('[1, 2,
3]'), negated field, number, object ('{ a: 1, b: 2 }'), string,
variable
```